

# Your frontend demo is 10% of the product. Here's how to build the other 90%.

You have a fully simulated React UI. Impressive for demos and investor pitches. Zero of it connects to real data, real sensors, or real money yet. This guide is your ordered sprint plan to turn it into a revenue-generating, factory-deployed SaaS in 16 weeks.

✓ WHAT YOU HAVE (the demo)

✓ React 19 + TypeScript frontend shell

✓ All UI components built (PowerFloorMap, SwiftSwitch, SavingsLedger etc.)

✓ SCADA visual design system

✓ PDF generation (pdfGenerator.ts)

✓ Urdu i18n + bilingual support

✓ Simulated telemetry (fake sensor data)

✓ Sound effects (Web Audio API)

✗ WHAT'S MISSING (production)

✗ Backend API server (zero backend exists)

✗ Authentication (anyone can see any factory)

✗ Real database (PostgreSQL + InfluxDB)

✗ Real-time MQTT sensor pipeline

✗ ML models (all

✗ WattBrain edge

✗ Multi-tenant f

✗ Billing & paym

✗ Production de

Frontend ✓ S0: Foundation S1: Auth S2: Pipeline S3: ML S4: Edge S5

## Understanding the Gap

Your React app renders beautiful dashboards using hardcoded simulated values inside each component. Every "real-time" reading is a `setInterval` randomizing numbers in local state. There is no server, no database, no real sensor. This is the gap you need to close — systematically, in this order.

LAYER 4 — PRESENTATION (✓ BUILT)

React 19 + TypeScript ✓

All UI Components ✓

Vite Build ✓

PDF Generator ✓

Urdu i18n ✓

LAYER 3 — CLOUD PLATFORM (✗ BUILD NEXT)

Go API Server X

Auth Service X

PostgreSQL X

InfluxDB X

Kafka X

ML FastAPI X

WebSocket Gateway X

Billing Service X

LAYER 2 – EDGE (X BUILD WEEK 9-10)

WattBrain Firmware X

MQTT Client X

Relay Control X

Offline Buffer X

OTA Updates X

LAYER 1 – HARDWARE (X PROCURE WEEK 3)

WattClamp™ CT Sensors X

RPi CM4 Controller X

RS-485 Bus X

4G LTE Module X

The Correct Build Order

Never skip ahead. Each sprint unlocks the next. The most common startup mistake is building ML before having a data pipeline — you end up with models that train on nothing. Follow this sequence exactly.

| Sprint            | Weeks | Goal                                              | Unlocks                       | Priority |
|-------------------|-------|---------------------------------------------------|-------------------------------|----------|
| S0 Foundation     | 1-2   | Monorepo, Docker Compose, CI/CD, domain           | Everything else               | BLOCKER  |
| S1 Auth + Tenancy | 3-4   | Login, JWT, multi-factory isolation, RBAC         | Can share with real customers | BLOCKER  |
| S2 Data Pipeline  | 5-6   | MQTT → Kafka → InfluxDB, WebSocket to frontend    | Real sensor data in UI        | BLOCKER  |
| S3 ML Models      | 7-8   | GOP, LoadShift MILP, Prophet baseline, Anomaly    | SwiftSwitch automation        | HIGH     |
| S4 Edge Firmware  | 9-10  | WattBrain OS, relay control, offline buffer       | Factory hardware deployable   | HIGH     |
| S5 Billing        | 11-12 | Savings ledger, invoicing, JazzCash/bank transfer | First revenue                 | HIGH     |

|              |       |                                                   |                                                                                             |        |
|--------------|-------|---------------------------------------------------|---------------------------------------------------------------------------------------------|--------|
| S6 Hardening | 13–14 | TLS everywhere, monitoring, load testing, backups | Production-grade reliability                                                                | MEDIUM |
| S7 Launch    | 15–16 | Deploy to AWS, first factory install, runbook     |  Revenue | LAUNCH |

SPRINT 0 · WEEKS 1–2

## Foundation: Monorepo, Docker & CI/CD

Before writing a single line of business logic, set up the scaffolding that will make every subsequent sprint fast and reliable. Do this once, correctly.

### Step 1 — Monorepo Structure

Your React app is one service. You will have 4–6 more. Use a monorepo so they share types, are versioned together, and deploy as one unit.

Recommended repo layout

```
wattwise/
├─ apps/
│   ├─ web/           # Your existing React app (move here)
│   ├─ api/           # Go backend (new)
│   ├─ ml/            # Python ML service (new)
│   └─ edge/          # WattBrain firmware (new)
├─ packages/
│   └─ shared-types/  # TypeScript + Go shared types (new)
├─ infra/
│   ├─ docker-compose.yml
│   ├─ terraform/     # AWS infrastructure as code
│   └─ k8s/           # Kubernetes manifests (Sprint 6)
├─ scripts/
│   └─ seed.sql        # Dev database seed data
└─ .github/
    └─ workflows/      # CI/CD pipelines
```

### Step 2 — Docker Compose (Local Dev Environment)

Every engineer on the team runs the entire platform locally with one command. No "works on my

machine" problems.

infra/docker-compose.yml

```
version: '3.9'
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: wattwise
      POSTGRES_USER: ww
      POSTGRES_PASSWORD: devpassword
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]

  influxdb:
    image: influxdb:2.7
    environment:
      DOCKER_INFLUXDB_INIT_MODE: setup
      DOCKER_INFLUXDB_INIT_ORG: wattwise
      DOCKER_INFLUXDB_INIT_BUCKET: sensors
      DOCKER_INFLUXDB_INIT_ADMIN_TOKEN: dev-token-change-in-prod
    ports: ["8086:8086"]

  mqtt:
    image: eclipse-mosquitto:2.0
    ports: ["1883:1883", "8883:8883"]
    volumes: [".:/infra/mosquitto.conf:/mosquitto/config/mosquitto.conf"]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  kafka:
    image: confluentinc/cp-kafka:7.6.0
    environment:
      KAFKA_KRAFT_MODE: "true"
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_NODE_ID: 1
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
    ports: ["9092:9092"]

volumes:
  pgdata:
```

```
$ docker compose up -d
```

```
# All services up in ~30 seconds. Run this from day 1.
```

## Step 3 — GitHub Actions CI/CD

.github/workflows/api.yml — Backend CI

```
name: API CI
on: [push, pull_request]
jobs:
  build-test:
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:16-alpine, env: { POSTGRES_PASSWORD: test } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with: { go-version: '1.22' }
      - run: cd apps/api && go build ./... && go test ./...
      - run: cd apps/api && go vet ./...
```

## Sprint 0 Checklist

- ☐ **Create GitHub monorepo** — move existing React app into `apps/web/`
- ☐ **Write docker-compose.yml** — PostgreSQL, InfluxDB, Mosquitto, Redis, Kafka
- ☐ **Initialize Go module** — `go mod init github.com/yourname/wattwise/api` in `apps/api/`
- ☐ **Set up GitHub Actions** — CI pipeline for web (typecheck), api (go test), ml (pytest)
- ☐ **Register domain** — `wattwise.pk` from PKNIC (~Rs. 2,500/yr). Point DNS to Cloudflare (free).
- ☐ **Create AWS account** — Enable ME-South-1 (Bahrain) region. Set up root MFA immediately.
- ☐ **Database migrations setup** — Install `golang-migrate`, write migration `001_initial_schema.sql`
- ☐ **Order hardware** — Place Alibaba order: 5x RPi CM4 4GB, 40x split-core CT sensors (100A), 5x RS-485 USB adapters



**Common mistake:** Don't start writing the Go API until docker-compose is running locally and migrations run clean. The foundation isn't glamorous but a broken local dev environment will cost you weeks of wasted debugging later.

SPRINT 1 · WEEKS 3-4

## Authentication, RBAC & Multi-Tenancy

This is the most critical security sprint. One bug here means Factory A can see Factory B's data — a catastrophic trust failure. Build it right the first time.

### Go API Server Bootstrap

apps/api/main.go

```
package main

import (
    "github.com/gin-gonic/gin"
    "github.com/wattwise/api/internal/auth"
    "github.com/wattwise/api/internal/factory"
    "github.com/wattwise/api/internal/telemetry"
    "github.com/wattwise/api/internal/middleware"
)

func main() {
    r := gin.New()
    r.Use(gin.Logger(), gin.Recovery(), middleware.CORS())

    // Public routes
    r.POST("/v1/auth/register", auth.Register)
    r.POST("/v1/auth/login", auth.Login)
    r.POST("/v1/auth/refresh", auth.Refresh)

    // Protected routes - JWT required
    api := r.Group("/v1", middleware.JWTAuth())
    {
        api.GET("/factories", factory.List)
        api.GET("/factories/:id/telemetry/live", telemetry.LiveHandler)
        api.GET("/factories/:id/predictions", predictions.Schedule)
        api.GET("/factories/:id/savings", savings.Get)
        api.GET("/ws/factories/:id", telemetry.WebSocket) // real-time
    }

    r.Run(":8080")
}
```

JWT Auth Flow

Access Token

Short-lived JWT (15 minutes). Signed with RS256 (asymmetric). Contains: user\_id, factory\_ids[], role. Stored in memory (not localStorage) on the frontend.

Refresh Token

Long-lived (7 days). Stored in HttpOnly cookie (not accessible to JavaScript). Rotated on every use. Revocation table in Redis. Used to silently refresh access tokens.

Multi-Tenant Data Isolation

Every database query MUST include the factory\_id from the JWT. Never trust the factory\_id from the URL parameter alone — it must match what's in the authenticated user's token. Use a middleware that enforces this.

internal/middleware/tenant.go — Tenant isolation enforcer

```
func RequireFactoryAccess() gin.HandlerFunc {
    return func(c *gin.Context) {
        claims := auth.GetClaims(c)
        factoryID := c.Param("id")

        // Verify the requested factory is in the user's allowed list
        if !claims.HasFactory(factoryID) {
            c.AbortWithStatusJSON(403, gin.H{"error": "access denied"})
            return
        }
        c.Set("factory_id", factoryID) // safe to use downstream
        c.Next()
    }
}
```

RBAC Roles

| Role        | Who                | Permissions                                      |
|-------------|--------------------|--------------------------------------------------|
| super_admin | WattWise team only | All factories, billing override, user management |

|                 |                     |                                                                 |
|-----------------|---------------------|-----------------------------------------------------------------|
| factory_owner   | Mill owner / CEO    | Own factories: full read, savings reports, billing, user invite |
| factory_manager | Operations manager  | Own factory: telemetry, automation controls, shift reports      |
| viewer          | Accountant, auditor | Own factory: read-only savings ledger and reports               |

Sprint 1 Checklist

- ☐ **Go API server bootstrapped** with Gin, structured logging (zerolog), graceful shutdown
- ☐ **PostgreSQL schema migrated** — factories, users, user\_factory\_access, sensor\_nodes tables
- ☐ **Auth endpoints working** — /register, /login, /refresh, /logout
- ☐ **JWT middleware** — RS256, tenant isolation enforced on all factory routes
- ☐ **Frontend wired to real auth** — Replace mock login in React with real API calls. Add AuthContext.
- ☐ **Factory CRUD API** — Create factory, add users, manage sensor node registry
- ☐ **Write auth integration tests** — Test: login returns token, wrong password fails, cross-tenant access returns 403

WattWise — Production Build Roadmap

SPRINT 2 · WEEKS 5-6

Real-Time Data Pipeline: MQTT → Kafka → InfluxDB → WebSocket

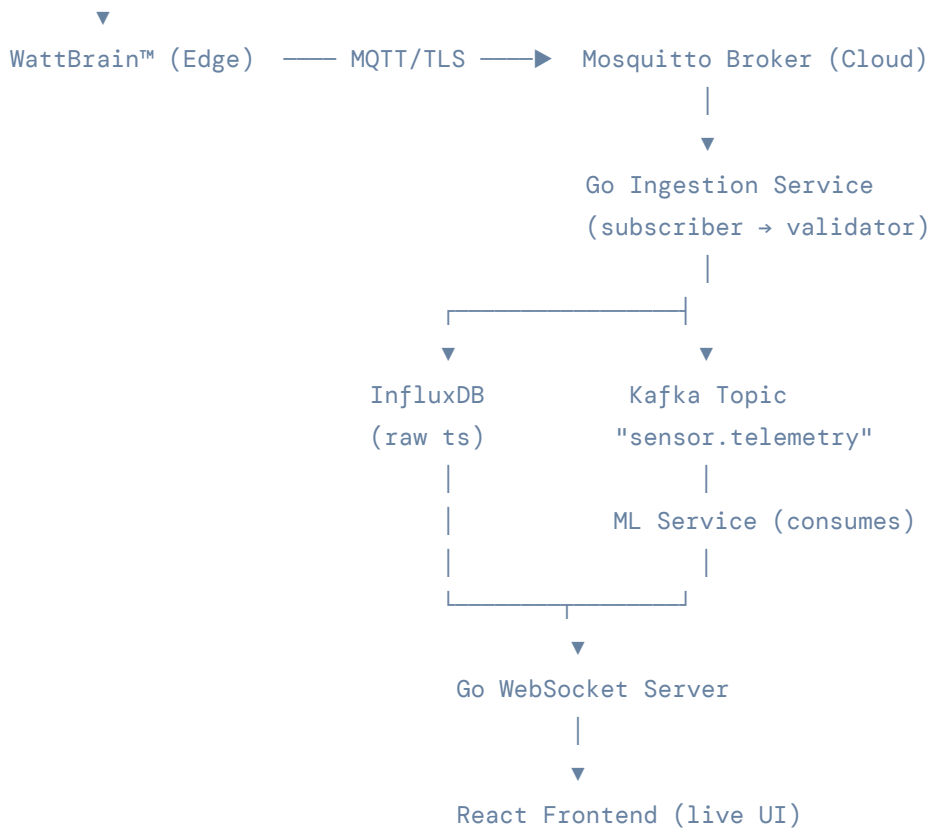
This sprint replaces your simulated `setInterval` data with real sensor readings flowing through a production-grade event pipeline. After this sprint, your frontend shows actual factory data.



**While hardware arrives:** Build a **WattBrain Simulator** — a Python script that publishes realistic MQTT packets (mimicking a real factory load cycle) so you can test the full pipeline before physical sensors are installed. This lets Sprint 2 and Sprint 4 run in parallel.

The Data Flow





## WattBrain Simulator (Build This First)

apps/edge/simulator/factory\_sim.py – Realistic load simulator

```

import paho.mqtt.client as mqtt
import json, time, random, math

FACTORY_ID = "fsd_mill_001"
BROKER      = "mqtt.wattwise.pk"
PORT        = 8883 # TLS port

SECTIONS = [
    {"id": "weaving_a", "base_kw": 310, "variance": 15},
    {"id": "dyeing_vats", "base_kw": 285, "variance": 20},
    {"id": "stenter", "base_kw": 142, "variance": 8},
    {"id": "compressors", "base_kw": 63, "variance": 5},
    {"id": "hvac", "base_kw": 47, "variance": 3},
]

def simulate_reading(section, on_grid=True):
    t = time.time()
    # Simulate realistic load curve: lower at shift start/end
    hour_factor = 0.8 + 0.2 * math.sin(t / 3600)
    kw = (section["base_kw"] + random.gauss(0, section["variance"])) * hour_factor
    return f
  
```

```

    return {
      "factory_id": FACTORY_ID,
      "node_id":    section["id"],
      "ts":         int(t * 1000), # milliseconds
      "power_kw":   round(kw, 2),
      "voltage_v":  round(random.gauss(220 if on_grid else 218, 2), 1),
      "freq_hz":    round(random.gauss(50.0 if on_grid else 50.1, 0.05), 2),
      "on_grid":    on_grid,
      "pf":         round(random.gauss(0.92, 0.02), 3),
    }

client = mqtt.Client()
client.tls_set() # TLS for production
client.connect(BROKER, PORT)
client.loop_start()

while True:
    for s in SECTIONS:
        payload = simulate_reading(s)
        topic = f"factory/{FACTORY_ID}/sensors/{s['id']}"
        client.publish(topic, json.dumps(payload), qos=1)
        time.sleep(5) # publish every 5 seconds

```

## Frontend: Replace Simulated Data

Your React components currently use `useState` + `setInterval` to fake real-time data. Replace this pattern with a custom WebSocket hook:

`apps/web/src/hooks/useFactoryTelemetry.ts`

```

import { useEffect, useRef, useState } from 'react'
import type { TelemetryPacket } from '@wattwise/shared-types'

export function useFactoryTelemetry(factoryId: string) {
  const [data, setData] = useState<TelemetryPacket | null>(null)
  const ws = useRef<WebSocket | null>(null)

  useEffect(() => {
    const token = getAccessToken() // from AuthContext
    ws.current = new WebSocket(
      `wss://api.wattwise.pk/v1/ws/factories/${factoryId}?token=${token}`
    )
    ws.current.onmessage = (e) => {
      setData(JSON.parse(e.data)) // replaces setInterval fake data
    }
    return () => ws.current?.close()
  }, [factoryId])
}

```

```
    }, [factoryId])

    return data
}
```

## Sprint 2 Checklist

- ☐ **Mosquitto broker running** with TLS cert (Let's Encrypt via Certbot). Port 8883 open.
- ☐ **Go MQTT subscriber** — subscribes to `factory/+/sensors/+`, validates packet, writes to Kafka
- ☐ **Kafka consumer → InfluxDB writer** — batch write every 1 second, 500 points/batch
- ☐ **WebSocket server** in Go — streams InfluxDB live queries to subscribed frontend clients
- ☐ **Factory simulator running** — Python script publishing realistic MQTT messages continuously
- ☐ **React components updated** — PowerFloorMap, SwiftSwitchSimulator, LoadShiftTimeline all consuming real WebSocket hook
- ☐ **WAPDA schedule scraper** — Python script scraping Faisalabad (FESCO) outage notices from wapda.gov.pk daily at 06:00 PKT

SPRINT 3 · WEEKS 7-8

## ML Models: From Simulated to Trained

Your UI already shows prediction outputs and savings calculations — but they're fake. This sprint trains and deploys the four real ML models. By the end, predictions come from actual sensor history.

### Model 1: Grid Outage Predictor (GOP) — Build This First

apps/ml/models/gop/train.py

```
import pandas as pd, xgboost as xgb
from sklearn.model_selection import cross_val_score
from sklearn.calibration import CalibratedClassifierCV
import onnx, skl2onnx

# Features: time-based + real-time signals
FEATURES = [
    'hour of day', 'day of week', 'is ramadan', 'is holiday'
```

```

    'feeder_outage_scheduled',    # from WAPDA scraper
    'voltage_v_lag5m',           # voltage 5 min ago
    'freq_hz_lag1m',             # frequency 1 min ago
    'voltage_rate_of_change',     # dV/dt — key early warning
    'national_deficit_mw',        # from NEPRA daily scrape
    'minutes_since_last_outage',  # outage clustering pattern
]

df = pd.read_parquet("data/fsd_feeder_A11_2yr_history.parquet")
X, y = df[FEATURES], df['outage_in_15min'] # binary label

model = xgb.XGBClassifier(
    n_estimators=300, max_depth=6,
    scale_pos_weight=8, # class imbalance: outages are ~12% of time
    eval_metric='aucpr' # precision-recall is the right metric here
)
calibrated = CalibratedClassifierCV(model, cv=5, method='isotonic')
calibrated.fit(X, y)

# Export to ONNX for WattBrain edge deployment
onnx_model = skl2onnx.convert_sklearn(calibrated, "GOP", initial_types=[...])
with open("models/gop_v1.onnx", "wb") as f: f.write(onnx_model.SerializeToString())

```

## Model 2: LoadShift MILP Optimizer

apps/ml/models/loadshift/optimizer.py

```

from ortools.sat.python import cp_model

def optimize_daily_schedule(factory_config, outage_windows):
    """
    Returns: ordered list of (process, start_slot, end_slot)
    Constraints: no process interrupted by outage window,
                dependency ordering respected, generator budget < 6h
    """
    model = cp_model.CpModel()
    slots = 48 # 30-minute slots across 24 hours

    for proc in factory_config.processes:
        start = model.NewIntVar(0, slots - proc.duration, f"start_{proc.id}")
        end = model.NewIntVar(proc.duration, slots, f"end_{proc.id}")
        model.Add(end == start + proc.duration)

        # Hard constraint: don't run critical processes during outage windows
        if proc.critical:

```

```

    for (ow_start, ow_end) in outage_windows:
        model.AddForbiddenAssignments([start], [[ow_start]])

# Minimize generator-hours (minimize sum of kW × on_generator_bool)
model.Minimize(generator_cost_objective)

solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = 3.0
solver.Solve(model)
return extract_schedule(solver, factory_config.processes)

```

### Model 3: Prophet Baseline (Critical for Billing)

This model must be locked-in at the start of each month and frozen. Never let it be retroactively adjusted. The baseline is legally the "before" state against which savings are calculated — it's the source of truth for your revenue.

apps/ml/models/baseline/estimator.py

```

from prophet import Prophet
import pandas as pd

def lock_monthly_baseline(factory_id: str, month: str) -> dict:
    """Run on 1st of each month at 00:01 PKT. Result is immutable."""

    # Pull 90 days of sensor data BEFORE this month
    df = fetch_influx_history(factory_id, lookback_days=90)
    df = df.rename(columns={'timestamp': 'ds', 'total_kw': 'y'})

    model = Prophet(
        seasonality_mode='multiplicative',
        yearly_seasonality=False,    # 90 days – no full year data
        weekly_seasonality=True,
        daily_seasonality=True,
        changepoint_prior_scale=0.05 # conservative – don't overfit
    )
    model.add_country_holidays(country_name='PK')
    model.fit(df)

    # Forecast what THIS month WOULD HAVE cost without WattWise
    future = model.make_future_dataframe(periods=30, freq='H')
    forecast = model.predict(future)
    projected_kwh = forecast['yhat'].sum()
    baseline_pkr = projected_kwh * current_tariff_rate(factory_id)

```

```
# Write to savings_records — APPEND ONLY, never UPDATE
return save_baseline(factory_id, month, baseline_pkr, audit_hash=hash_dataframe(df))
```

## FastAPI Model Serving

apps/ml/main.py — ML Service (internal only, not public)

```
from fastapi import FastAPI
app = FastAPI()

# GOP: called by Go API for prediction endpoint
@app.get("/predict/outage/{factory_id}")
async def predict_outage(factory_id: str):
    features = await fetch_live_features(factory_id)
    prob = gop_model.predict_proba([features])[0][1]
    return {"outage_probability": prob, "trigger_automation": prob > 0.85}

# LoadShift: called daily at 05:00 PKT via cron
@app.post("/schedule/{factory_id}")
async def generate_schedule(factory_id: str):
    outages = await get_todays_predicted_outages(factory_id)
    schedule = optimize_daily_schedule(factory_config, outages)
    await save_schedule(factory_id, schedule)
    return schedule
```

## Sprint 3 Checklist

- ☐ **Collect 30+ days simulator data** in InfluxDB before training (run simulator from Sprint 2)
- ☐ **GOP v1 trained and serving** — XGBoost + calibrated probabilities, ONNX exported
- ☐ **LoadShift MILP optimizer working** — Google OR-Tools, <3s solve time for 20 processes
- ☐ **Prophet baseline running** — Cron job locks baseline on 1st of each month
- ☐ **Anomaly detector running** — Isolation Forest on 5-minute rolling windows, alert on deviation  $> 3\sigma$
- ☐ **FastAPI ML service deployed** in Docker — internal only (not public-facing)
- ☐ **Frontend LoadShiftTimeline connected** to real schedule from MILP optimizer (not hardcoded)

# Edge Firmware: WattBrain Production OS

Hardware should have arrived by now. This sprint turns a Raspberry Pi CM4 into a deployable WattBrain unit — reading real CT sensors, running local inference, and controlling relays.

## Edge OS Setup

```
# 1. Flash Ubuntu 22.04 Server (ARM64) to CM4 eMMC via rpiboot
$ sudo rpiboot
$ sudo dd if=ubuntu-22.04-arm64.img of=/dev/sda bs=4M status=progress

# 2. After first boot, run WattBrain provisioning script
$ curl -fsSL https://edge.wattwise.pk/install.sh | sudo bash

# This script: installs Python 3.11, ONNX Runtime, Mosquitto client,
# downloads factory-specific config, registers device, starts systemd services
```

## Core Edge Services (systemd)

| Service          | Description                                                                               | Restart Policy |
|------------------|-------------------------------------------------------------------------------------------|----------------|
| wattbrain-sensor | Polls CT sensors via Modbus RS-485 every 100ms, writes to ring buffer                     | always         |
| wattbrain-infer  | Runs ONNX GOP model every 30s on rolling window. Outputs relay commands.                  | always         |
| wattbrain-relay  | Receives relay commands, controls GPIO contactor outputs with safety interlocks           | always         |
| wattbrain-mqtt   | Publishes buffered readings to cloud MQTT every 5s. Queues offline if no connectivity.    | always         |
| wattbrain-ota    | Polls edge.wattwise.pk/updates every hour. Downloads and applies signed firmware updates. | on-success     |



**Critical safety requirement:** The relay control service MUST have a hardware watchdog. If wattbrain-relay crashes or hangs for >30 seconds, the watchdog must trigger a GPIO fail-safe that defaults all relay outputs to OPEN (grid-connected). Never leave a factory in an ambiguous switchover state due to a software crash.

## OTA Update System

You will have sensors physically inside factories you cannot easily visit. A robust Over-The-Air update system is not optional — it's how you fix bugs, push model improvements, and patch security vulnerabilities across your entire fleet without visiting each site.

OTA update flow (simplified)

```
# WattBrain polls hourly
GET https://edge.wattwise.pk/v1/devices/{device_id}/updates
→ { "version": "1.4.2", "url": "...", "sha256": "...", "signature": "..." }

# If newer version:
1. Download to /tmp/update.tar.gz
2. Verify SHA-256 hash
3. Verify Ed25519 signature against WattWise public key
4. Extract to /opt/wattbrain-staging/
5. Run health check: wattbrain-sensor passes 60s smoke test
6. Atomically swap /opt/wattbrain/ → new version
7. Report success to cloud. Rollback if health check fails.
```

## Sprint 4 Checklist

- ☐ **Ubuntu flashed on RPi CM4** — SSH accessible, static IP configured
- ☐ **Modbus RS-485 reading CT sensors** — Verify ampere readings match a clamp meter
- ☐ **ONNX Runtime running on-device** — GOP model inference in <200ms on CM4
- ☐ **GPIO relay control working** — Test relay click/response with multimeter, verify safety interlocks
- ☐ **SQLite ring buffer** — 72h of readings stored locally, FIFO eviction, verified with connectivity drop test
- ☐ **MQTT client publishing** — Data flowing cloud → InfluxDB from physical hardware (not simulator)
- ☐ **OTA update server built** — Can push and roll back firmware remotely
- ☐ **Hardware watchdog configured** — BCM2835 hardware watchdog, 30-second timeout, fail-safe relay default

SPRINT 5 · WEEKS 11-12

## Billing: Savings Ledger → Real Money

Your SavingsLedger.tsx shows beautiful numbers. This sprint makes them collect actual payment in Pakistani Rupees.

### B2B Payment Options (Pakistan)

**Bank Transfer (preferred for B2B)** — Issue invoice, customer pays via IBFT (Inter-Bank Fund Transfer). Reconcile via HBL/Meezan internet banking API or Finja Connect. No MDR fees. Standard for factories.

**JazzCash Business** — For smaller or first payments. 1–1.5% MDR. Quick to integrate.

**Stripe (PKR disabled)** — Do NOT use Stripe for PKR collections. Use for USD invoices to foreign buyers only.

### Invoice Generation

Your pdfGenerator.ts already generates ESCO certificates. Extend it to generate **tax invoices** in Pakistani format: NTN of both parties, Sales Tax Registration (if applicable), HSN code for IT services, invoice number per FBR serial requirements.

For gain-share model: invoice monthly on the 5th. Net-15 payment terms. Late payment: 1.5%/month markup clause in the customer agreement.

## Savings Ledger — Make It Append-Only at DB Level

PostgreSQL — Enforce immutability with a trigger

```
-- Prevent any UPDATE or DELETE on savings_records
CREATE RULE savings_no_update AS ON UPDATE TO savings_records
DO INSTEAD NOTHING;

CREATE RULE savings_no_delete AS ON DELETE TO savings_records
DO INSTEAD NOTHING;

-- Audit log: track every read of savings records (for dispute resolution)
CREATE TABLE savings_audit_log (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  record_id   UUID REFERENCES savings_records(id),
  accessed_by UUID REFERENCES users(id),
  accessed_at TIMESTAMPTZ DEFAULT NOW(),
  ip_address  INET
```

## Sprint 5 Checklist

- ☐ **Savings ledger service** — Monthly cron: locks baseline, calculates actual cost, writes append-only record with SHA-256 hash
- ☐ **Invoice PDF generator** — FBR-compliant tax invoice (NTN, Sales Tax Reg No., serial numbers)
- ☐ **Bank transfer reconciliation** — HBL or Meezan business API to match incoming IBFT payments to invoices
- ☐ **Customer billing portal** — React page: past invoices, payment status, download ESCO certificates
- ☐ **Automated payment reminder** — Email + WhatsApp message (Twilio) on Day 10 and Day 14 if unpaid
- ☐ **DB immutability enforced** — PostgreSQL rules prevent UPDATE/DELETE on savings\_records
- ☐ **Customer agreement e-signing** — DocuSign or HelloSign (or simple PDF + scan accepted for early customers)

SPRINT 6 • WEEKS 13-14

## Production Hardening: Security, Monitoring & Reliability

### Security Checklist

- ☐ **TLS everywhere** — MQTT 8883, HTTPS API, WSS WebSocket, DB connections. Let's Encrypt via Caddy.
- ☐ **Rate limiting** — 100 req/min per IP on auth endpoints. 1,000 req/min on API. Redis-backed.
- ☐ **Input validation** — Validate all sensor packets (range checks, timestamp bounds, factory\_id whitelist)
- ☐ **Secrets manager** — AWS Secrets Manager for all credentials. No .env files in production.
- ☐ **Penetration test** — Run OWASP ZAP on API. Fix all HIGH findings before launch.
- ☐ **Database encryption at rest** — Enable RDS encryption. InfluxDB volume encrypted (AWS EBS).

### Monitoring Stack

- ☐ **Grafana dashboard** — API latency P50/P95/P99, Kafka lag, InfluxDB write throughput, error rates
- ☐ **Sensor dropout alert** — PagerDuty alert if a factory's sensor goes silent >5 minutes (could mean WattBrain offline)
- ☐ **Sentry error tracking** — Both React frontend and Go backend. Set up before first customer.
- ☐ **Uptime monitoring** — UptimeRobot or Pingdom on API endpoint. SMS alert to CTO if down >2 min.
- ☐ **Database backups** — Daily PostgreSQL dump to S3. Weekly InfluxDB snapshot. Test restore quarterly.
- ☐ **Load test** — k6 script simulating 50 simultaneous factories, 10k MQTT messages/minute. Fix bottlenecks.

SPRINT 7 • WEEKS 15-16

## Go Live: First Factory Deployment



**Target milestone:** By Week 16, you have one real factory in Faisalabad with physical WattClamp sensors installed, WattBrain running, data flowing to cloud, and the factory manager logged into the dashboard on their phone. Not perfect — but real. This is worth more than any further simulated feature.

### Week 15 — Soft Launch (Shadow Mode)

- ☐ **Production AWS infra deployed** — ECS Fargate for API + ML, RDS Aurora PostgreSQL, InfluxDB Cloud, ElastiCache Redis
- ☐ **wattwise.pk live** — React app served via CloudFront CDN. HTTPS. Mobile-responsive.
- ☐ **First pilot factory signed** — Customer agreement signed. Install date confirmed.
- ☐ **Hardware installed** — WattBrain + 10–15 CT clamps. Shadow mode (monitoring only, no automation).
- ☐ **Runbook written** — How to respond to: sensor offline, billing dispute, switchover failure, cloud outage

### Week 16 — Full Activation

- ☐ **30-day shadow report delivered** — Show factory owner what they spent vs. what WattWise would have saved. The data closes the deal.

- ☐ **SwiftSwitch automation enabled** — Factory owner gives sign-off. First automated relay switchover logged.
- ☐ **First month's savings tracked** — Baseline locked on 1st. Actual cost tracked throughout month.
- ☐ **First invoice issued** — 20% of verified savings. Customer pays. 🦉 First WattWise revenue.
- ☐ **Ask for referral** — Happy factory owner introduces you to 2 peers at APTMA meeting.

## Reference: Complete Repo Structure

```

wattwise/                                # GitHub monorepo root
|
├─ apps/
|   └─ web/                              # React 19 + Vite (your existing code)
|       └─ src/
|           └─ components/                # Your existing components
|           └─ hooks/
|               └─ useFactoryTelemetry.ts # NEW: real WebSocket hook
|           └─ lib/
|               └─ api.ts                 # NEW: typed API client (fetch wrapper)
|               └─ auth.ts                # NEW: AuthContext + JWT handling
|               └─ pages/
|                   └─ Login.tsx          # NEW: real auth form
|           └─ package.json
|       |
|   └─ api/                              # Go backend (NEW)
|       └─ cmd/server/main.go
|       └─ internal/
|           └─ auth/                      # JWT, RBAC, refresh tokens
|           └─ factory/                   # Factory CRUD, tenant isolation
|           └─ telemetry/                 # WebSocket, InfluxDB queries
|           └─ ingestion/                 # MQTT subscriber, Kafka producer
|           └─ savings/                   # Ledger, invoice generation
|           └─ middleware/                # CORS, rate limiting, tenant check
|       └─ go.mod
|       |
|   └─ ml/                               # Python ML service (NEW)
|       └─ models/
|           └─ gop/train.py               # Grid Outage Predictor
|           └─ loadshift/optimizer.py
|           └─ baseline/estimator.py

```

```

| | | └─ anomaly/detector.py
| | └─ serving/main.py          # FastAPI serving layer
| | └─ requirements.txt
| |
| └─ edge/                      # WattBrain firmware (NEW)
|   └─ wattbrain/
|     └─ sensor_bus.py          # Modbus RS-485 CT reader
|     └─ inference.py           # ONNX local inference
|     └─ relay_ctrl.py          # GPIO relay control + watchdog
|     └─ mqtt_client.py         # Publish to cloud MQTT
|     └─ local_store.py         # SQLite ring buffer
|     └─ simulator/factory_sim.py # Dev simulator
|     └─ install.sh             # OTA provisioning script
|
└─ packages/
  └─ shared-types/              # TypeScript types + OpenAPI spec
    └─ src/telemetry.ts
    └─ src/factory.ts
  |
└─ infra/
  └─ docker-compose.yml         # Local dev: all services
  └─ mosquitto.conf
  └─ terraform/
    └─ main.tf                  # ECS, RDS, ElastiCache, MSK
    └─ variables.tf
  └─ k8s/                       # Sprint 6+
  |
└─ .github/
  └─ workflows/
    └─ web.yml                  # TypeScript check + build
    └─ api.yml                  # Go test + lint
    └─ ml.yml                    # pytest + model validation

```

## Reference: Environment Variables

apps/api/.env.example — Never commit real values

```

# Database
DATABASE_URL=postgres://ww:password@localhost:5432/wattwise?sslmode=require
INFLUXDB_URL=http://localhost:8086
INFLUXDB_TOKEN=your-token-here
INFLUXDB_ORG=wattwise
INFLUXDB_BUCKET=wattwise

```

```
INFLUXDB_BUCKET=SENSORS
```

```
# Auth
```

```
JWT_PRIVATE_KEY_PATH=/secrets/jwt_private.pem
```

```
JWT_PUBLIC_KEY_PATH=/secrets/jwt_public.pem
```

```
JWT_ACCESS_TTL_MINUTES=15
```

```
JWT_REFRESH_TTL_DAYS=7
```

```
# MQTT
```

```
MQTT_BROKER_URL=mqtts://mqtt.wattwise.pk:8883
```

```
MQTT_CLIENT_CERT=/secrets/server.crt
```

```
MQTT_CLIENT_KEY=/secrets/server.key
```

```
# Kafka
```

```
KAFKA_BROKERS=localhost:9092
```

```
KAFKA_TOPIC_SENSOR=sensor.telemetry
```

```
# Cache
```

```
REDIS_URL=redis://localhost:6379
```

```
# ML Service (internal)
```

```
ML_SERVICE_URL=http://ml-service:8000
```

```
# Notifications
```

```
TWILIO_ACCOUNT_SID=ACxxxxxxx
```

```
TWILIO_AUTH_TOKEN=xxxxxxx
```

```
TWILIO_WHATSAPP_FROM=+14155238886
```

```
# Pakistan SMS (Jazz/Telenor)
```

```
JAZZ_SMS_API_KEY=xxxxxxx
```

```
JAZZ_SENDER_ID=WattWise
```

## Reference: Production Docker Build

apps/api/Dockerfile — Multi-stage Go build (12MB final image)

```
# Stage 1: Build
```

```
FROM golang:1.22-alpine AS builder
```

```
WORKDIR /app
```

```
COPY go.mod go.sum ./
```

```
RUN go mod download
```

```
COPY . .
```

```
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-w -s" -o wattwise-api ./cmd/server
```

```
# Stage 2: Minimal runtime (12MB image)
FROM gcr.io/distroless/static-debian12
COPY --from=builder /app/wattwise-api /wattwise-api
EXPOSE 8080
USER nonroot:nonroot
ENTRYPOINT ["/wattwise-api"]
```

apps/web/Dockerfile — React static build + Nginx

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN npm i -g pnpm && pnpm install --frozen-lockfile
COPY . .
RUN pnpm build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

# Reference: AWS Monthly Cost Map

| Service               | Config                     | Cost (USD/month) | Notes                               |
|-----------------------|----------------------------|------------------|-------------------------------------|
| ECS Fargate (API)     | 2 tasks x 0.5 vCPU / 1GB   | ~\$25            | Auto-scales to 4 tasks at peak      |
| ECS Fargate (ML)      | 1 task x 1vCPU / 2GB       | ~\$30            | ML service — add GPU at Series A    |
| RDS Aurora PostgreSQL | db.t4g.small, Multi-AZ     | ~\$55            | Upgrade to medium at 100+ customers |
| InfluxDB Cloud        | Usage-based (1B writes/mo) | ~\$40            | Grows with sensor count             |
| MSK Kafka             | kafka.t3.small x 2         | ~\$65            | Managed Kafka — no ops overhead     |
| Elasticache Redis     | cache.t4g.micro            | ~\$15            | Session + rate limiting             |

| Elasticache Redis        | cache.t4g.micro | ~\$10     | Session state limiting                 |
|--------------------------|-----------------|-----------|----------------------------------------|
| S3 (backups + artifacts) | 100GB storage   | ~\$5      | Lifecycle to Glacier after 90 days     |
| CloudFront (CDN)         | React app       | ~\$5      | Free tier covers 1TB/mo                |
| Mosquitto (EC2)          | t3.small        | ~\$15     | Self-managed MQTT broker               |
| CloudWatch Logs          | 50GB/mo         | ~\$10     | Set retention to 30 days               |
| TOTAL (bootstrap)        |                 | ~\$265/mo | ~Rs. 74,000/mo — covered by 1 customer |



**AWS Pakistan note:** Apply for AWS Activate (Startup tier) — gives USD 1,000–5,000 in free credits for new startups. Enough to cover infrastructure for 4–18 months before first revenue. Apply via your incubator (LUMS NIC, Invest2Innovate, NUST NSTP) for higher-tier vouchers.